

Sayan Ghosh¹

General Electric Research,
Niskayuna, NY 12309
e-mail: sayan.ghosh1@ge.com

Govinda Anantha Padmanabha

Department of Aerospace and Mechanical
Engineering,
University of Notre Dame,
Notre Dame, IN 46556
e-mail: ganantha@nd.edu

Cheng Peng

Department of Aerospace and Mechanical
Engineering,
University of Notre Dame,
Notre Dame, IN 46556
e-mail: cpeng2@nd.edu

Valeria Andreoli

General Electric Research,
Niskayuna, NY 12309
e-mail: valeria.andreoli@ge.com

Steven Atkinson

General Electric Research,
Niskayuna, NY 12309
e-mail: steven@atkinson.mn

Piyush Pandita

General Electric Research,
Niskayuna, NY 12309
e-mail: piyush.pandita@ge.com

Thomas Vandeputte

General Electric Research,
Niskayuna, NY 12309
e-mail: vandeput@ge.com

Nicholas Zabarar

Department of Aerospace and Mechanical
Engineering,
University of Notre Dame,
Notre Dame, IN 46556
e-mail: nzabarar@nd.edu

Liping Wang

General Electric Research,
Niskayuna, NY 12309
e-mail: wangli@ge.com

Inverse Aerodynamic Design of Gas Turbine Blades Using Probabilistic Machine Learning

One of the critical components in industrial gas turbines (IGT) is the turbine blade. The design of turbine blades needs to consider multiple aspects like aerodynamic efficiency, durability, safety, and manufacturing, which make the design process sequential and iterative. The sequential nature of these iterations forces a long design cycle time, ranging from several months to years. Due to the reactionary nature of these iterations, little effort has been made to accumulate data in a manner that allows for deep exploration and understanding of the total design space. This is exemplified in the process of designing the individual components of the IGT, resulting in a potential unrealized efficiency. To overcome the aforementioned challenges, we demonstrate a probabilistic inverse design machine learning (PMI) framework to carry out an explicit inverse design. PMI calculates the design explicitly without excessive costly iteration and overcomes the challenges associated with ill-posed inverse problems. In this study, the framework will be demonstrated on inverse aerodynamic design of three-dimensional turbine blades.
[DOI: 10.1115/1.4052301]

Keywords: inverse design, invertible neural network, machine learning, probabilistic modeling, Gaussian process, gas turbine, aerodynamic design, data-driven design, design automation, design methodology, generative design

1 Introduction

The design of industrial gas turbines (IGTs) is a multidisciplinary, complex process that often requires numerous iterations across different teams to achieve multiple design objectives. Every subsystem of the turbine must be designed considering various

multidisciplinary targets like performance, manufacturability, temperature limits, component life, simplicity of assembling, and disassembling procedures. Often, design objectives and constraints conflict and numerous iterations at system level (firing temperature, pressure ratio, engine flow, etc.), module level (compressor, combustor, turbine, etc.), and component level (individual airfoils) are required to achieve a cross-functionally valid design. Many of these iterations occur serially, starting from an aerodynamic assessment, followed by heat transfer and mechanical assessments with several internal design loops within and between each discipline. The sequential nature of these iterations

¹Corresponding author.

Contributed by the Design Automation Committee of ASME for publication in the JOURNAL OF MECHANICAL DESIGN. Manuscript received May 1, 2021; final manuscript received August 25, 2021; published online September 21, 2021. Assoc. Editor: Yi Ren.

forces long design cycle times, ranging from several months to years.

An inverse design process aims to positively impact the difficulties that encumber the traditional iterative process. In inverse design, the performance target is set as an input to the process, while the design parameters become the output. Using this approach, a design is explicitly generated from target performance metrics. Over the last few decades, various approaches of inverse design have been developed for a range of applications. These methods can be broadly classified into two categories: (a) an optimization-based approach and (b) a direct inverse design approach [1–5]. The former is similar to forward design methods, whereas the latter is used to find a design that satisfies all constraints and achieves the target performance. Most of the methods used for turbomachinery inverse designs are computational fluid dynamics (CFD)-based algorithms, where the flow fields information is used to guide the blade redesign to match a given design target [6–8]. Recent work on inverse design has focused on developing frameworks that allow the user to quantify uncertainty around the optimal user inputs or design parameters for multiple applications, see Refs. [9–12]. Specifically, work treating problems in the turbomachinery domain includes the work of Refs. [13,14], where the frameworks proposed leverage state-of-the-art optimization techniques to solve for the optimal input. The basic gap is the inability of the frameworks to combine and generate data from multiple sources under a limited budget and explicit inverse mapping that allows one to generate “x” as a function of “y.”

With recent advances in the area of *deep learning*, new methods have revolutionized the inverse design process [15–22]. However, the current methods struggle with high-dimensional design spaces, require large amounts of high-fidelity training data, and have difficulty dealing with problems that are ill-posed or ill-conditioned.

In this work, we demonstrate a scalable framework for explicit inverse design, named probabilistic machine learning for inverse design of aerodynamic systems (PMI) [23], which overcomes these challenges to enable inverse aerodynamic design of turbine blade airfoils. Given some desired performance quantities, the inverse problem is solved explicitly by sampling from the PMIs

explicit inverse mapping, making it trivial to characterize the marginal posterior density over potential designs, which captures both the fundamental ill-posed nature of the inverse problem and the reality of epistemic uncertainty induced by limited resources to query the underlying physics of the system.

The designs of interest in this work are turbomachinery components that are applicable to not only IGTs but also to aviation turbine engines, wind turbines, and hydro turbines. As a representative problem that demonstrates the capabilities of the PMI framework, we will focus on the inverse design of the last stage blade (LSB) cross-sectional airfoils. As the largest rotating component in the IGT, the LSB is one of the most mechanically challenging components to design and often determines the total power output of the machine. The increasing push for larger and hotter turbines to drive down the cost of electricity has resulted in increasingly challenging LSB designs. With deference typically skewed toward durability, this has generally resulted in greater aerodynamic compromises that negatively impact blade efficiency. This also drives longer design cycles as designers incrementally search for acceptable aero-mechanical solutions. Methods that enable reduction in design cycle time while also improving aerodynamic efficiency are therefore highly desirable. In the next section, we will give brief details of PMI framework. The following section includes the demonstration of our PMI framework on a toy problem. In Sec. 4, we demonstrate the framework on inverse aerodynamic design of 3D blade of last stage blade of IGT. Finally, we will present conclusions and discussion in Sec. 5.

2 Probabilistic Inverse Design Machine Learning Framework

The PMI framework, as shown in Fig. 1, generates an explicit functional representation of the inverse design process. The main element of this framework is a conditional invertible neural network (cINN). The cINN is used to represent designs explicitly for the desired targets of performance and constraints. The framework entails the following two-step process:

- (1) Forward modeling: efficient modeling of the forward process

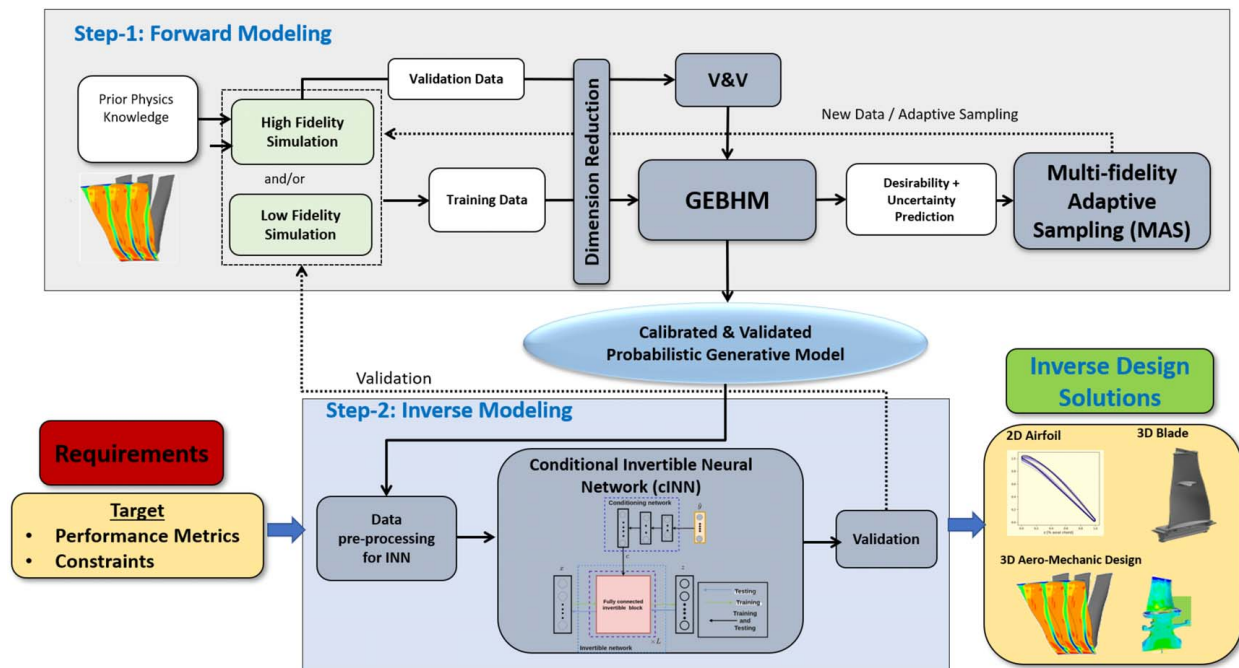


Fig. 1 Probabilistic machine learning for inverse design of aerodynamic systems framework

- (2) Inverse modeling: training a cINN using the data generated in the first step.

In the forward modeling step, a probabilistic multifidelity Gaussian process (MFGP) regression model for the expensive experiments is constructed using the GE Bayesian hybrid modeling (GEBHM) [24,25]. To reduce the cost associated with the design of the computer experiments [26–30] required by the GEBHM, a multifidelity adaptive sampling [26] is used to adaptively determine the experiment and the level of fidelity that are needed to enhance the performance. The data generated in step 1 using the surrogate of the forward model (MFGP) will be used to train the cINN in step 2.

In the inverse modeling step, a cINN is trained, which is a parameterized Bayesian bijective transform between the input or the design variables and the performance characteristics or the engineering quantities of interest (QoI). The invertible block of cINN is built using a flow-based conditional generative model. One of the key characteristics of the architecture used is its ability to capture the uncertainty due to the forward process as well as the epistemic uncertainty due to limited data, allowing engineers to carry out a trade-off between cost and risk. To solve an inverse problem for a given target performance or QoI, the cINNs explicit inverse mapping can be trivially sampled to generate potential designs while capturing the fundamental ill-posed nature of the inverse problem as well as the epistemic uncertainty induced by limited resources. One of the uniqueness of the work presented here is the aspect of the different modules working in conjunction to enable explicit inverse mappings.

2.1 Forward Modeling. In the first step of the framework, we train a probabilistic surrogate model of the forward process using Gaussian process (GP) regression [31]. Depending on the available sources of the data, we build either a SF GP or an MFGP.

2.1.1 Single-Fidelity GP. Consider a GP surrogate model of the form:

$$y(\mathbf{x}) \sim GP(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')) \quad (1)$$

where $m(\mathbf{x})$, assumed to be zero here, is the mean function, and $k(\mathbf{x}, \mathbf{x})$ is the covariance function for a vector valued input denoted by $\mathbf{x}$ of dimension d . In this study, the covariance function is assumed to be the squared exponential kernel [31]:

$$k(\mathbf{x}, \mathbf{x}) = \sigma^2 \exp(-\beta(\mathbf{x} - \mathbf{x})^2) + I\lambda^2 \quad (2)$$

where β are the (inverse) length scale parameters collected in a vector, one per input dimension, σ^2 captures the data variance as the amount of data variance captured by the model and λ^2 quantifies the amount of variance captured by the residuals. This GP models an output $y(\mathbf{x})$ given an input vector $\mathbf{x}$. Observe now a set of inputs and outputs and collect these in a training data set of N elements $D = \{x_i, y_i\}_{i=1}^N$.

The GP fitting process translates to fitting the hyperparameters associated with the matrix:

$$K_{(i,j)} = k(x_i, x_j) \quad (3)$$

where x_i is the i th training datum.

The hyperparameters of the GP are defined as the vector $\phi = (\sigma, \beta, \lambda)$ and need to be fitted to the training data set D . In this work, priors are placed on the hyperparameters to incorporate the initial belief into the data modeling before seeing the data itself, such as smoothness. It can be inferred from above that for our problems of learning the model's hyperparameters, it is always a problem of estimating $m \times (d + 2) + 1$ number of parameters. The likelihood of observing the training data for a selected set θ of hyperparameters is $L(\mathbf{D}|\phi) = \frac{1}{|\Sigma|^{\frac{1}{2}}} \exp(-\frac{1}{2} \mathbf{Y}^T \Sigma^{-1} \mathbf{Y})$, where the i , j th element of the matrix $\mathbf{Y}$ is the j th output value for the i th training

data point. The conditional posterior of the hyperparameter set ϕ can then be written as $p(\phi|\mathbf{D}) \propto L(\mathbf{D}|\phi) \prod_{k=1}^m p(\phi^{(k)})p(\lambda)p(\sigma)$. This expression, commonly known as the *target distribution*, is known only up to a proportionality constant. By combining the priors with the likelihood function, the problem of fitting ϕ boils down to sampling from the extrema of the posterior distribution $p(\phi|\mathbf{D})$ since, of course, larger values of $p(\phi|\mathbf{D})$ implies more likely models ϕ . The Markov Chain Monte Carlo (MCMC) method both seeks the extrema and provides a way to sample from it, in cases where the normalization constant of the posterior probability distribution is unknown.

2.1.2 Multifidelity Gaussian Process. In the industrial setting, computational budget allocation might be limited to only a handful of the afforded expensive runs of a high-fidelity simulation code. Conversely, access to simplified models (low fidelity) may provide useful information that at least can capture the general trend of the high-fidelity model. The MFGP surrogate can be trained to bridge the information from various levels of the model's complexity. To train the MFGP, we follow Kennedy OHagans (KOH) methodology [32] where the observed data (i.e., the high-fidelity data), $y(\mathbf{x})$, are represented as a linear combination of a low fidelity and model a discrepancy term [32]:

$$y(\mathbf{x}) = \eta(\mathbf{x}; \theta) + \delta(\mathbf{x}) + \epsilon \quad (4)$$

where θ are calibration parameters, i.e., parameters of the low-fidelity model that may or may not have a physical meaning, which can be tuned to better match the observed data $y(\mathbf{x})$. Note that Eq. (4) contains of two separate GPs and a term ϵ that represents additive Gaussian noise with zero mean and a constant variance. Namely, $\eta(\mathbf{x}, \theta)$ is a GP for the simulator data or the low-fidelity data and $\delta(\mathbf{x})$ is a GP for capturing the discrepancy between the simulator (or the low-fidelity data) and the observed data (or the high-fidelity data), which are collected at the independent variable locations. Also, we will not consider any calibration parameters (θ) with low-fidelity data in the current work, i.e., $\eta(\mathbf{x}, \theta) = \eta(\mathbf{x})$.

Importantly, each GP in Eq. (4) is fitted to its own data set. For example, the low-fidelity model $\eta(\mathbf{x})$ is fitted to a data set $D_\eta = \{z_i, w_i\}_{i=1}^{N_\eta}$, where z is the independent variable and w is the dependent variable (output from the computer simulator). The discrepancy $\delta(\mathbf{x})$ GP is fitted by using information from both the low- and the high-fidelity data $D_y = \{x_i, y_i\}_{i=1}^{N_y}$. The KOH method is a two-part solution: build a base model of the low-fidelity data and a discrepancy model that maps the low-fidelity model to high-fidelity data.

Generally, z and $\mathbf{x}$ are not located at the same points and most typically will not have the same size, i.e., the simulator (low-fidelity model) is run at different input points than the observed (high-fidelity data), but nothing prevents them from being the same.

The covariance matrix on a finite sample of points from both the simulator and the real-world experiment is now given by the following overall structure (the subscript mf refers to multifidelity):

$$K_{mf} = \begin{bmatrix} K_y & 0 & 0 \\ 0 & K_u & K_{uw} \\ 0 & K_{uw}^T & K_w \end{bmatrix} \quad (5)$$

where each covariance matrix has been labeled with a subscript identifying which dependent variable data are being modeled. K_y is the covariance matrix of the high-fidelity data, K_w is the covariance of the low-fidelity data, and the newly introduced variable u refers to the low-fidelity data predicted on the high-fidelity points, and thus, K_{uw} is the covariance matrix between the low-fidelity data and the low-fidelity model predicting high-fidelity data. This covariance matrix contains multiple hyperparameters via the covariance matrices, which in turn are defined from the covariance function in Eq. (2). Similar to single-fidelity GP, the hyperparameters of MFGP are estimated using MCMC, see the study by Ghosh et al. [24] for more details, which is a fully Bayesian approach to estimate

the posterior distribution of the parameters based on the observed data and prior distribution.

2.2 Inverse Modeling. The inverse problem aims to recover the high-dimensional input $\mathbf{x} \in \mathbb{R}^M$ given noisy and gappy data $\tilde{\mathbf{y}} \in \mathbb{R}^D$. This problem is ill posed since one may not be able to uniquely recover the high-dimensional input $\mathbf{x}$ given the noisy observations $\tilde{\mathbf{y}}$. To address this challenge, we develop a model that maps the given noisy observations $\tilde{\mathbf{y}}$ to the unknown high-dimensional input space $\mathbf{x}$. In this work, we use a deep generative model for constructing the inverse surrogate model.

2.2.1 Conditional Invertible Neural Network. Recently, several conditional deep generative models were developed using generative adversarial networks [33] and variational autoencoders (VAEs) [34] for solving the computer vision problems [35–37]. The main drawbacks of these conditional deep generative models are that it is hard to train the model stably, and it is also difficult to obtain samples with sharp features [38]. Therefore, in this work, we extend the real-valued nonvolume preserving (real NVP) [39] architecture by a conditioning network that basically conditions the observations $\tilde{\mathbf{y}}$ to the real NVP [39] architecture. The main advantage of using the real NVP is that it explicitly learns the data distribution with exact log-likelihood evaluation and allow stable training.

The real NVP architecture transforms a simple distribution $p(\mathbf{z})$ to a complex distribution $p(\mathbf{x})$ using a sequence of affine coupling layers. Given the latent-variable $\mathbf{z} \sim p(\mathbf{z})$, the input space $\mathbf{x} \sim p(\mathbf{x})$ can be inferred as $\mathbf{x} = f(\mathbf{z})$ and the latent variable can also be calculated inversely with $\mathbf{z} = f^{-1}(\mathbf{x})$. Here, the transformation function f is invertible, and it is referred to as the affine coupling layer that should satisfy the following properties: (a) the transformation must be invertible and (b) the Jacobian determinant should be easy to compute, and finally, the dimensions of the input and the output must be the same. The real NVP consists of forward and inverse propagation. During the forward propagation, the model transforms the input $\mathbf{x}$ to the latent space $\mathbf{z}$ as follows. Given an M dimensional input $\mathbf{x}$ and $m < M$, the output of an affine coupling layer $\mathbf{z}$ follows:

$$\begin{aligned} \mathbf{z}_{1:m} &= \mathbf{x}_{1:m} \\ \mathbf{z}_{m+1:M} &= \mathbf{x}_{m+1:M} \odot \exp(s(\mathbf{x}_{1:m})) + t(\mathbf{x}_{1:m}) \end{aligned} \quad (6)$$

During the inverse propagation, the model transforms the latent space $\mathbf{z}$ to the input space $\mathbf{x}$ as follows:

$$\begin{aligned} \mathbf{x}_{1:m} &= \mathbf{z}_{1:m} \\ \mathbf{x}_{m+1:M} &= (\mathbf{z}_{m+1:M} - t(\mathbf{z}_{1:m})) \odot \exp(-s(\mathbf{z}_{1:m})) \end{aligned} \quad (7)$$

where the scale $s(\cdot)$ and shift $t(\cdot)$ networks map from $\mathbb{R}^m$ to $\mathbb{R}^{M-m}$ and $\odot$ denotes an element-wise multiplication. The Jacobian of the transformation function f conducted by an affine coupling

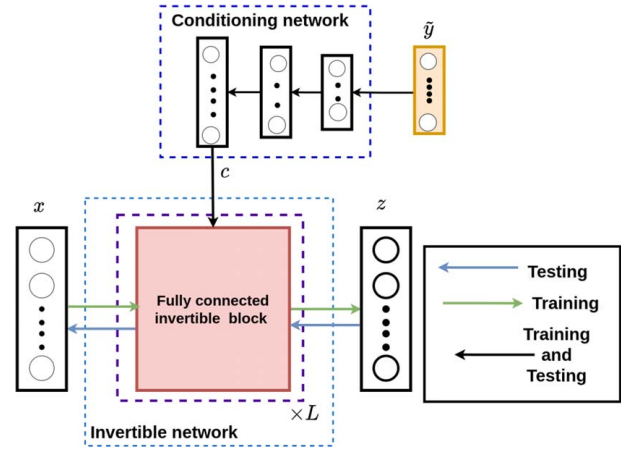


Fig. 2 cINN: the training process is shown in forward arrow and the testing process is shown in backward arrow

layer is derived as following [39]:

$$J = \begin{bmatrix} \mathbb{I}_m & \mathbf{0}_{m \times (M-m)} \\ \frac{\partial \mathbf{z}_{m+1:M}}{\partial \mathbf{x}_{1:m}} & \text{diag}(\exp(s(\mathbf{x}_{1:m}))) \end{bmatrix} \quad (8)$$

Hence, the determinant of the Jacobian is simply the product of diagonal elements. For complete details on the real NVP architecture, we direct the reader to the work by Dinh et al. [39].

2.2.2 Network Details. Both the invertible and the conditioning networks are constructed using fully connected layers. Here, the conditioning network takes in the noisy observations $\tilde{\mathbf{y}}$ as the input and the output being the conditioning inputs $\mathbf{c}$. The invertible network consists of a sequence of affine coupling layers with L blocks that transforms the input space $\mathbf{x}$ to the latent space, $\mathbf{z}$ with the conditioning inputs $\mathbf{c}$ from the conditioning network during forward propagation. Here, during forward propagation, we denote $\{\mathbf{x}'_{l-1}\}_{l=1}^L \in \mathbb{R}^M$ as the input to each invertible blocks and the output for that invertible block is $\{\mathbf{x}'_l\}_{l=1}^L \in \mathbb{R}^M$. During the inverse propagation, the model transforms the latent space $\mathbf{z}$ to the input space $\mathbf{x}$ conditioned on the conditioning inputs $\mathbf{c}$. Here, we denote $\{\mathbf{z}'_{l-1}\}_{l=1}^L$ as the input and the output as $\{\mathbf{z}'_l\}_{l=1}^L$ for that invertible block during inverse propagation. Figure 2 illustrates our inverse surrogate model showing the training (forward propagation) and the testing (inverse propagation) process.

The concept of concatenating the conditioning inputs to the invertible network blocks are similar to the work of Padmanabha and Zabarar [40], Geneva and Zabarar [41], Zhu et al. [42], and Ardizzone et al. [38]. However, we modify the conditioning and the invertible architectures to construct an inverse surrogate model that maps the noisy observations to the input space. The

Table 1 Forward and inverse propagation of each affine coupling layer conditioned on the conditioning input $\mathbf{c}$

Forward	Inverse
$\bar{\mathbf{x}}_{1,l-1}, \bar{\mathbf{x}}_{2,l-1} = \text{split}(\mathbf{x}'_{l-1})$ $\hat{\mathbf{x}}_{1,l-1} = \text{concat}(\bar{\mathbf{x}}_{1,l-1}, \mathbf{c})$ $(s, t) = \text{AffineCouplingNet}(\hat{\mathbf{x}}_{1,l-1})$ $\bar{\mathbf{x}}_{2,l} = \exp(s) \odot \bar{\mathbf{x}}_{2,l-1} + t$ $\bar{\mathbf{x}}_{1,l} = \bar{\mathbf{x}}_{1,l-1}$ $\mathbf{x}'_l = \text{concat}(\bar{\mathbf{x}}_{1,l}, \bar{\mathbf{x}}_{2,l})$	$\bar{\mathbf{z}}_{1,l-1}, \bar{\mathbf{z}}_{2,l-1} = \text{split}(\mathbf{z}'_{l-1})$ $\hat{\mathbf{z}}_{1,l-1} = \text{concat}(\bar{\mathbf{z}}_{1,l-1}, \mathbf{c})$ $(s, t) = \text{AffineCouplingNet}(\hat{\mathbf{z}}_{1,l-1})$ $\bar{\mathbf{z}}_{2,l} = (\bar{\mathbf{z}}_{2,l-1} - t) / \exp(s)$ $\bar{\mathbf{z}}_{1,l} = \bar{\mathbf{z}}_{1,l-1}$ $\mathbf{z}'_l = \text{concat}(\bar{\mathbf{z}}_{1,l}, \bar{\mathbf{z}}_{2,l})$

Note: During forward propagation, the input is $\{\mathbf{x}'_{l-1}\}_{l=1}^L \in \mathbb{R}^M$ and the output is $\{\mathbf{x}'_l\}_{l=1}^L \in \mathbb{R}^M$. During inverse propagation, the input is the latent space $\{\mathbf{z}'_{l-1}\}_{l=1}^L$ and the output is $\{\mathbf{z}'_l\}_{l=1}^L$. Here, the split operation divides the data into two parts: $1 < m < M$, such that $\bar{\mathbf{x}}_{1,l-1} = \mathbf{x}'_{(1:m),\{l-1\}}$ and $\bar{\mathbf{x}}_{2,l-1} = \mathbf{x}'_{(m+1:M),\{l-1\}}$ during the forward operation. Similarly, $\bar{\mathbf{z}}_{1,l-1} = \mathbf{z}'_{(1:m),\{l-1\}}$ and $\bar{\mathbf{z}}_{2,l-1} = \mathbf{z}'_{(m+1:M),\{l-1\}}$ for the inverse operation. Here, AffineCouplingNet represents the scale $s(\cdot)$ and shift $t(\cdot)$ networks.

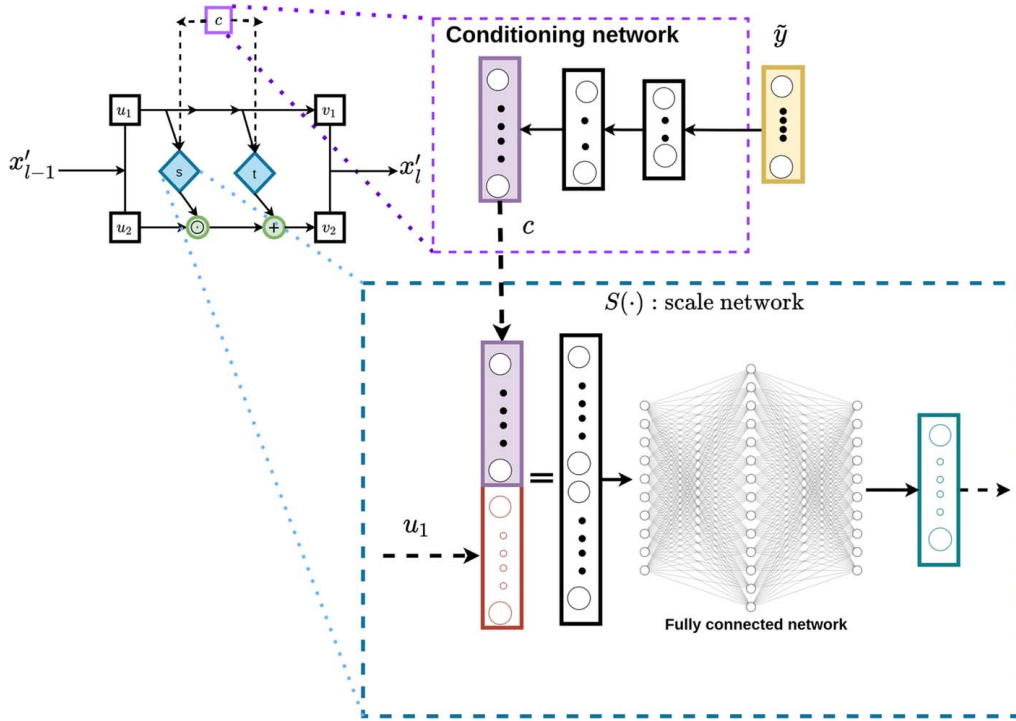


Fig. 3 Illustration of a scale network, $s(\cdot)$ in an invertible block. Here, $u_1 = \mathbf{x}'_{(1:m),\{l-1\}}$, $u_2 = \mathbf{x}'_{(m+1:M),\{l-1\}}$, $v_1 = \mathbf{x}'_{(1:m),\{l\}}$, and $v_2 = \mathbf{x}'_{(m+1:M),\{l\}}$.

details of forward and inverse propagation of each affine coupling block can be found in Table 1. Figure 3 illustrates the single affine conditional block where the conditioning inputs c from the conditioning network and the input x_{l-1} for the invertible block l are considered as the input to the scale network $s(\cdot)$. Similarly, we consider the same input for shift $t(\cdot)$ network. In this work, both the scale and the shift networks are fully connected layers with LeakyReLU [43] activation function between each hidden layer. In this work, we empirically perform an extensive search for network architecture designs such as the number of affine coupling layers, and the numbers of hidden layers, and an extensive hyperparameters search such as the learning rate, weight decay, and the number of epochs that work well for this inverse problem.

To clarify, the invertible network or the block of the cINN architecture is the sequence of affine coupling layers (one shown in Fig. 2), which maps the input space $\mathbf{x}$ to latent space $\mathbf{z}$. To account for the observation $\mathbf{y}$ during training (and target objective during the inverse processing), a conditioning network is used, which maps $\mathbf{y}$ to a conditioning input c . The conditioning input c becomes input to the scale $s(\cdot)$ and the shift $t(\cdot)$ network as shown in Fig. 3.

Loss function: Given a set of training data $\mathcal{D} = \{\mathbf{x}^i, \tilde{\mathbf{y}}^i\}_{i=1}^N$, we consider the *maximum a posteriori* (MAP) estimate of the model parameters θ :

$$\theta^* = \arg \max_{\theta} \prod_{i=1}^N p(\theta | \mathbf{x}^i, \tilde{\mathbf{y}}^i) \quad (9)$$

In this work, the invertible and the conditional networks are trained in an end to end fashion. Therefore, the model parameters, θ , include the invertible network parameters θ_l and the conditioning network parameters θ_c , i.e., $\theta = [\theta_c, \theta_l]$. We further simplify Eq. (9) as follows:

$$\theta^* = \arg \max_{\theta} p(\theta) \prod_{i=1}^N p(\mathbf{x}^i | \tilde{\mathbf{y}}^i, \theta) \quad (10)$$

where $p(\theta)$ is the prior on the model parameters and the conditional likelihood is simplified as follows. First, we introduce a bijection $f_{\theta}(\cdot)$ that is parametrized by both the invertible and the conditioning model parameters $\theta = [\theta_c, \theta_l]$ and conditioned on the noisy observations $\tilde{\mathbf{y}}$, that maps the input space $\mathbf{x}$ to the latent space $\mathbf{z} \sim p(\mathbf{z})$. Next, using the change of variables formula, we obtain the following conditional likelihood:

$$p_{\theta}(\mathbf{X} | \tilde{\mathbf{Y}}) = \prod_{i=1}^N \left[p(f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})) \cdot \left| \det \left(\frac{\partial f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})}{\partial \mathbf{x}^{(i)}} \right) \right| \right] \quad (11)$$

where $\mathbf{X} = \{\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}\}$, $\tilde{\mathbf{Y}} = \{\tilde{\mathbf{y}}^{(1)}, \tilde{\mathbf{y}}^{(2)}, \dots, \tilde{\mathbf{y}}^{(N)}\}$ and the latent variable $\mathbf{z}^{(i)} = f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})$.

The conditional log-likelihood $\log p_{\theta}(\mathbf{X} | \tilde{\mathbf{Y}})$ can be exactly evaluated as follows:

$$\log p_{\theta}(\mathbf{X} | \tilde{\mathbf{Y}}) = \sum_{i=1}^N \log p(f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})) + \log \left| \det \left(\frac{\partial f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})}{\partial \mathbf{x}^{(i)}} \right) \right| \quad (12)$$

One can rewrite the MAP estimate in Eq. (10) can be written as the minimizing the below loss function:

$$\mathcal{L} = - \sum_{i=1}^N \left[\log p(f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})) + \log \left| \det \left(\frac{\partial f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})}{\partial \mathbf{x}^{(i)}} \right) \right| \right] - \log p(\theta) \quad (13)$$

Since we consider a standard normal distribution for the latent space, we can further simplify the $\log p(f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)}))$ as $-\frac{\|f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})\|_2^2}{2}$.

As mentioned earlier, we consider a Gaussian prior on the model parameter with mean 0 and variance σ_{θ}^2 . Further, simplifying

Eq. (13), we obtain the following loss function:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left[\frac{\|f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)})\|_2^2}{2} - \log |J_i| \right] + \tau \|\theta\|_2^2 \quad (14)$$

where $\tau = 1/2\sigma_{\theta}^2$ and the Jacobian determinant $J_i = \det\left(\frac{\partial(f_{\theta}(\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)}))}{\partial \mathbf{x}^{(i)}}\right)$. The training process is enumerated in Algorithm 1.

Algorithm 1 Training procedure for the inverse surrogate model

Input: Training data: $\{\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)}\}_{i=1}^N$, number of epochs: E_{train} , learning rate: η , mini-batch size: Q , and number of flow model: L .

for $epoch = 1$ to E_{train} **do**

 Sample a minibatch from the training dataset:

$\{\mathbf{x}^{(i)}, \tilde{\mathbf{y}}^{(i)}\}_{i=1}^Q$ and pass the observations $\{\tilde{\mathbf{y}}^{(i)}\}_{i=1}^Q$ to the conditioning network to obtain the conditioning inputs $\{\mathbf{c}^{(i)}\}_{i=1}^Q : \mathbf{c}^{(i)} = g_{\theta_c}(\tilde{\mathbf{y}}^{(i)})$ and the input $\{\mathbf{x}^{(i)}\}_{i=1}^Q$ to the conditional invertible network:

for $l = 1$ to L **do**

$\mathbf{x}_0^{(i)} = \mathbf{x}^{(i)}$

$\mathbf{x}_l^{(i)}, \mathbf{J}_l^{(i)} = f_{l,\theta_l}(\mathbf{x}_{l-1}^{(i)}, \mathbf{c}^{(i)}) \quad \triangleright f$ is the invertible network that includes permutation.

$\hat{\mathbf{z}}^{(i)} = \{\mathbf{x}_L^{(i)}\}$

 Compute $\mathbf{J}_{\text{final}}^{(i)} = \sum_{l=1}^L (\mathbf{J}_l^{(i)})$

$\mathcal{L} = \text{Loss}(\hat{\mathbf{z}}^{(i)}, \mathbf{J}_{\text{final}}^{(i)})$

$\nabla \theta \leftarrow \text{Backprop}(\mathcal{L})$

$\theta \leftarrow \theta - \eta \nabla \theta$

Output: Trained cINN network.

Inversion: After the training process, we obtain the inverse solutions to the cINN as illustrated in Algorithm 2. Given the observation $\tilde{\mathbf{y}}$ as the input to the conditioning network, we get the conditioning input $\mathbf{c}$ to the invertible network. For the invertible network, we first generate $S = 1000$ samples: $\mathbf{Z} = \{\mathbf{z}^{(j)}\}_{j=1}^S$, where $\mathbf{z}$ follows normal distribution as $\mathbf{z}^{(j)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and then we obtain the input field $\{\hat{\mathbf{x}}^{(j)}\}_{j=1}^S$ using invertible network conditioned on the observations $\tilde{\mathbf{y}}$:

$$\hat{\mathbf{x}}^{(j)} = f_{\theta}^{-1}(\mathbf{z}^{(j)}, \tilde{\mathbf{y}}) \quad (15)$$

Algorithm 2 Inverse solution: conditional invertible neural networks

Input: Trained invertible and conditioning networks, observations: $\tilde{\mathbf{y}}$, number of samples: S , and number of flow model: L .

$\mathbf{z}^{(j)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$; $\mathbf{Z} = \{\mathbf{z}^{(j)}\}_{j=1}^S$.

$\tilde{\mathbf{y}} = \text{tile}(\tilde{\mathbf{y}})$; $\tilde{\mathbf{y}} = \{\tilde{\mathbf{y}}^{(j)}\}_{j=1}^S \quad \triangleright \text{tile: constructs a new array by repeating } \tilde{\mathbf{y}}$

Pass the observations $\{\tilde{\mathbf{y}}^{(j)}\}_{j=1}^S$ to the conditioning network to obtain the conditioning inputs $\{\mathbf{c}^{(j)}\}_{j=1}^S : \mathbf{c}^{(j)} = g_{\theta_c}(\tilde{\mathbf{y}}^{(j)})$ and the samples $\{\mathbf{z}^{(j)}\}_{j=1}^S$ to the invertible network:

for $l = 1$ to L **do**

$\mathbf{z}_0^{(j)} = \mathbf{z}^{(j)}$

$\mathbf{z}_l^{(j)} = f_{l,\theta_l}^{-1}(\mathbf{z}_{l-1}^{(j)}, \mathbf{c}^{(j)}) \quad \triangleright f^{-1}$ is the inverse mapping that includes permutation

$\hat{\mathbf{x}}^{(j)} = \mathbf{z}_L^{(j)}$

Output: Samples: $\{\hat{\mathbf{x}}^{(j)}\}_{j=1}^S$

3 Demonstration Example: Toy Function

We now consider the application of Algorithms 1 and 2 on a simple low-dimensional case. We consider an input domain $\Omega_x = [-L_x/2, L_x/2]^{d_x}$ equipped with a uniform probability density $p(\mathbf{x})$ over Ω_x . The ground truth function is modeled as $y = f(\mathbf{x}) + \epsilon$, where $f(\cdot)$ is a simple quadratic,

$$f(\mathbf{x}) = (\mathbf{W}\mathbf{x} - \boldsymbol{\mu})^T(\mathbf{W}\mathbf{x} - \boldsymbol{\mu}) \quad (16)$$

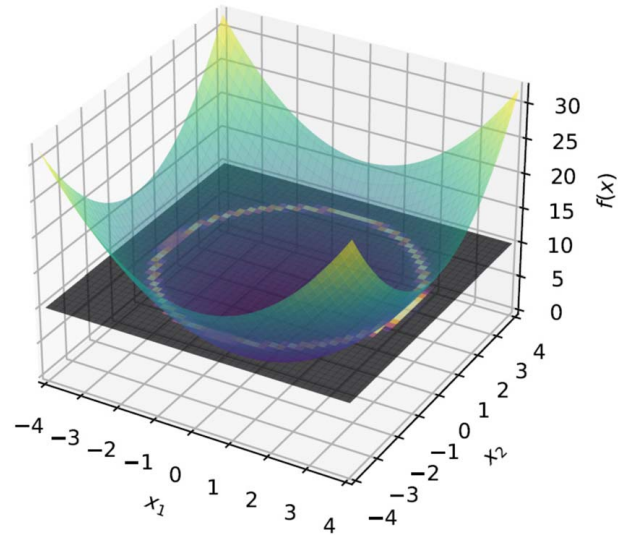


Fig. 4 Invertible neural network (INN) toy example: plot of the two-dimensional function along with a level set at $y = 10$

and $\epsilon \sim \mathcal{N}(0, 0.5^2)$. Equation (16) corresponds to a second-order approximation of the neighborhood of any local extremum in a twice-differentiable function via appropriate choice of the affine transformation parameters $\mathbf{W}$ and $\boldsymbol{\mu}$. This makes it relevant to many engineering design and optimization tasks, where one is interested in locating such an extremum in design space (maximizing efficiency, minimizing weight, etc.). For our example, we choose $d_x = 2$ input dimensions, $L_x = 4$, $\mathbf{W} = \mathbf{I}_{d_x \times d_x}$, and $\boldsymbol{\mu} = \mathbf{0}_{d_x}$. This means that the unique global minimum of $f(\mathbf{x})$ is $\mathbf{x} = \mathbf{0}$ with value $y = 0$. Furthermore, the set of inputs consistent with any $y \geq 0$ is the circle of radius $y^{1/2}$ centered at the origin.

An illustration of $f(\cdot)$ with a slice at $y = 10$ showing the ground truth inverse probability $p(\mathbf{x} | y)$ convolved with a mild Gaussian smoother (for visualization purposes) is shown in Fig. 4. We see that, physically, the *true* conditional distribution that we wish to capture with the cINN is ring shaped. Intuitively, *any* $\mathbf{x}$ on this ring results in the specified y , reflecting the fundamental one-to-many nature associated with the inverse problem in general. In other words, the marginal densities $p(x_i | y)$ will *not* necessarily converge to having zero standard deviation with sufficiently many data.

We train a cINN with $L = 4$ affine coupling pairs using Algorithm 1 with minibatches of $Q = 128$ data produced in an online setting by sampling $\mathbf{x} \sim p(\mathbf{x})$, $y = f(\mathbf{x}) + \epsilon$. A flat weighting function is used. We optimize for 2×10^4 iterations using the Adam optimizer [44] with cosine-annealed learning rate from $\eta = 3 \times 10^{-3}$ to $\eta = 10^{-5}$ by the last iteration.

With the trained model, a designer can query the cINN with different target objectives (y) to generate designs solutions $\mathbf{x}$. Heat maps of the conditional distribution $p(\mathbf{x} | y)$ learned by the cINN are plotted in Fig. 5 for various values of y . All the samples from the distributions are possible design for corresponding design target. We see that the cINN has successfully learned to model the expected conditional distribution, in particular one-to-many solution of the toy problem. Because the cINN explicitly models these conditional distributions, we can trivially solve any design problem, i.e., find any number of designs $\mathbf{x}$ that are consistent with an arbitrary desired output y . By contrast, a traditional forward modeling-based approach would require many repeated queries to implicitly determine even a single $\mathbf{x}$ such that $f(\mathbf{x}) = y$. From the designer's perspective, one may have multiple design choices that satisfy the performance requirement. One can then chose the final design based on additional preference, desirability toward other requirements or constraints.

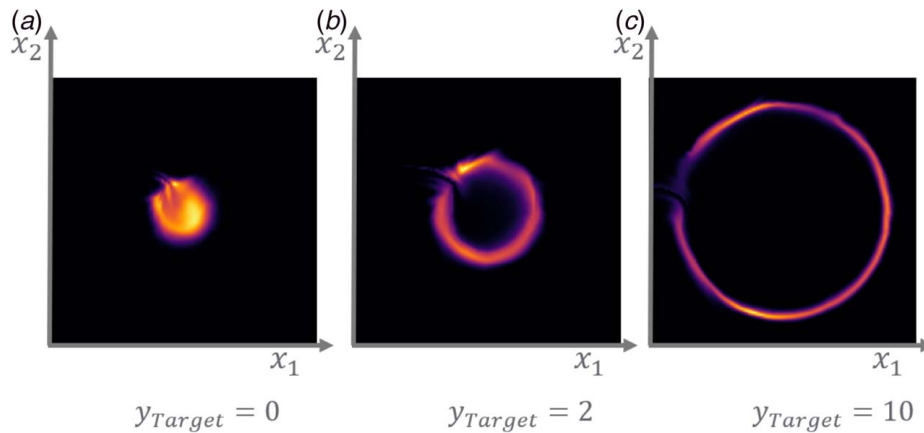


Fig. 5 INN toy example: conditional distributions learned by the model for (a) $y = 0$, (b) $y = 2$, and (c) $y = 10$

4 Inverse Aerodynamic Design of Three-Dimensional Turbine Blades

In this section, we will present the process of inverse aerodynamic design of 3D turbine last stage blade.

4.1 Modeling and Simulations. The reference design and the turbine operating conditions for modeling and simulation of 3D blade are based on an industrial gas turbine last stage. Turbine blade performance is evaluated using steady-state Reynolds-averaged Navier–Stokes (RANS) CFD at two different fidelity levels: a fast running coarse mesh for broader design space exploration and a slower running fine mesh for accuracy refinement. The 3D blade surface is constructed with seven 2D airfoil profiles at different spanwise locations from hub to tip. Figure 6 shows a view of the full 3D blade and a 2D sectional plane represented by an airfoil. Each airfoil section is characterized by 12 independent parameters, as previously shown in Fig. 7. These parameters allow independent control of the stagger angle, leading and trailing edge metal angles, leading edge diameter, suction and pressure side wedge angles, leading edge and trailing edge metal angles, and the airfoil curvature between the leading and trailing edges. The ranges for these parameters are selected to provide a wide design space while respecting geometrical constraints.

The 2D sections are aligned relative to each other in circumferential and axial space by aligning the section centers of gravity (CG) along a radial line through the hub section CG (referred to as the

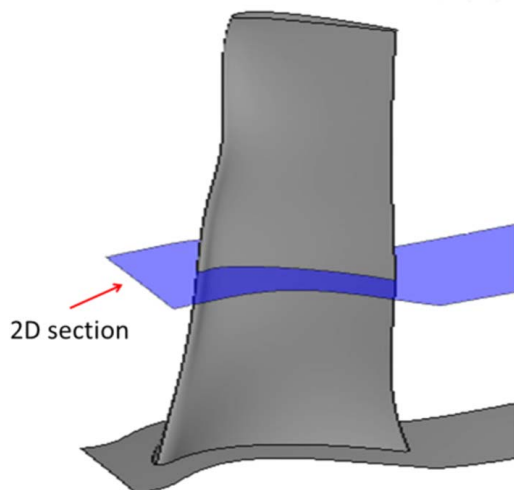


Fig. 6 Construction of 3D airfoil based on 2D sections

stacking line). After the section CGs are aligned, one additional parameter, referred to as the airfoil lean angle, is applied to reorient the stacking line relative to the radial direction. Surfaces are fit through the seven stacked sections to create the full, continuous, 3D airfoil definition. Eighty-five total parameters are therefore required to define the complete 3D airfoil shape.

The parameters are expressed as offsets from a baseline, requiring that each section starts from an appropriate reference design. An in-house software package tailored specifically for turbomachinery design uses the parameters described earlier to create the airfoil coordinates, and these coordinates are then transformed into a full 3D computer-aided design model of the rotor blade for the CFD grid generation. A 3D structured mesh is built using a commercially available software package. Grid templates are built from the baseline geometry and used consistently for all the cases throughout the optimization. With this approach, all the grids have similar refinement and quality metrics. To simulate the full stage, the upstream stator is included in the CFD calculation for each case. The design of the vane and the vane mesh are not altered through the optimization.

The 3D CFD analysis is performed using GE's in-house CFD solver TACOMA, a second-order accurate (in time and space), finite-volume, block-structured, and compressible flow solver. The steady RANS calculations are solved with a mixing plane between rotating and stationary components. Source terms are included at various locations along the end walls to simulate the injected cooling, leakage, and purge flows. The multifidelity dataset is based on two sets of grids, a low-resolution (low node count) grid and a high-resolution grid, to obtain low- and high-fidelity data, respectively. Several grids are assessed to down select coarse and fine grid resolutions that provide a sufficiently wide trade-off between fidelity and computational cost. A production-level grid is selected to provide accurate performance metrics for the high-cost data. The grid is then progressively relaxed to reduce computational cost for the low-cost cases. Figure 8 shows the ideal Mach number at the tip for the baseline geometry, assessed using the coarse and the fine grid, i.e., low and high fidelity, respectively. The ideal Mach number distributions agree very well at all locations, except for the rear portion of the suction side. Due to the poorer resolution of the coarse grid, the shock is not accurately captured in the low-fidelity calculations. Considering all operations involved in the assessment of each case, starting from the selection of the parameters to the calculation of the objectives, the cost ratio between high- and low-fidelity simulations is 4.5–6.0. Some operations, like geometry generation and postprocessing, can be considered approximately grid independent.

Two main objectives are selected as metrics of the 3D blade's performance. These scalar targets are calculated from the CFD results and will be used as inputs in the inverse design approach:

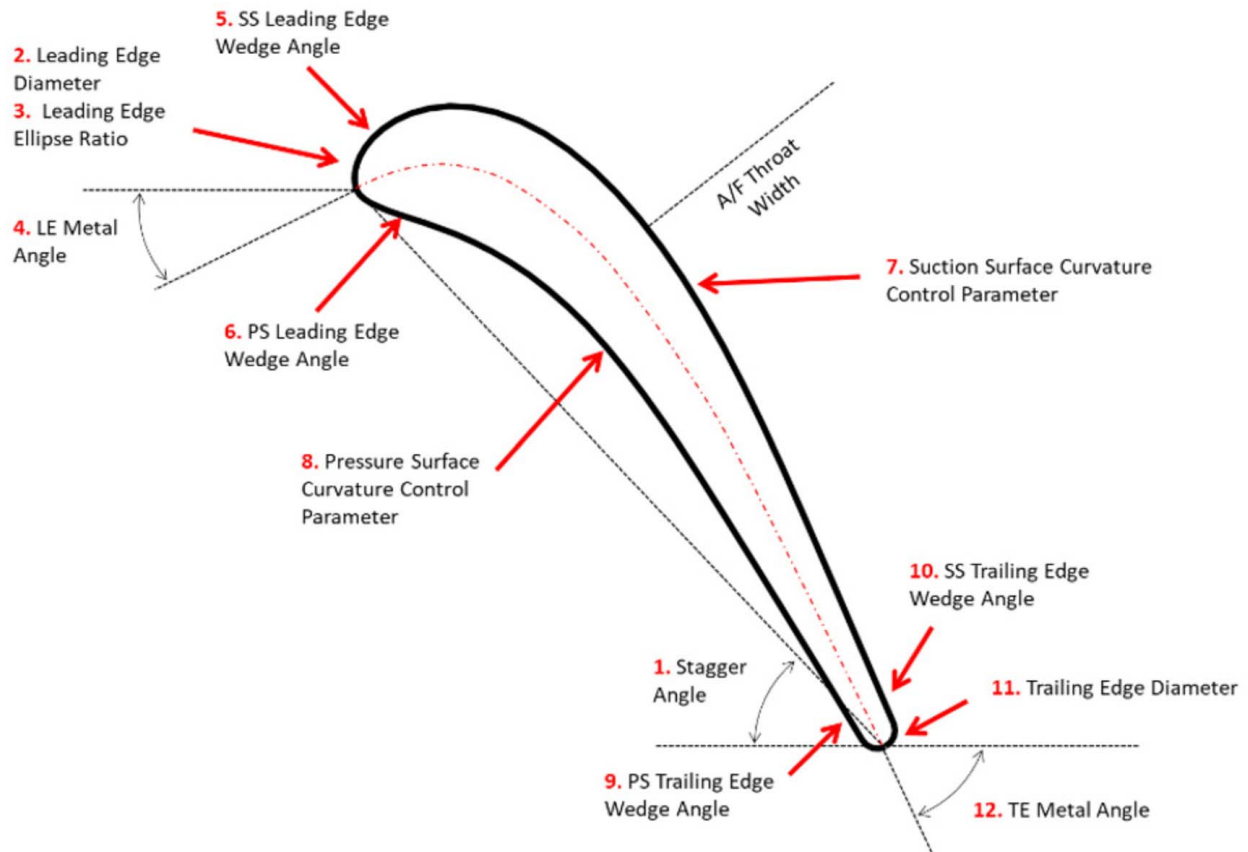


Fig. 7 Two-dimensional airfoil parametrization

- Scalar objective 1: Aerodynamic efficiency. The efficiency is calculated as the ratio of mechanical and ideal power. All the inputs required to calculate the efficiency value are directly available as output quantities of the CFD simulation. Generally, in a design process, the preference is to maximize efficiency.
- Scalar objective 2: Pseudo-reaction or degree of reaction. In a stage calculation, the degree of reaction indicates the split of flow acceleration between stator and rotor. The pseudo-reaction is calculated for each geometry and is monitored to account for the changes in turbine operating condition due to the blade shape. The design preference is normally to target a baseline value.

The two main objectives, efficiency and degree of reaction, are plotted in Fig. 9, using the data from initial rounds of DOE. The

cases assessed with a low-fidelity grid are represented with circles, while the cases assessed with high fidelity are represented with diamonds. The square represents the performance of the baseline design. Figure 9 shows that the range of outputs generated by low-fidelity analysis are very similar to high-fidelity analysis and are correlated. Although low-fidelity analysis is not very accurate, it stores information that can be used by the MFGP model to build accurate model by trading off cost with accuracy.

In addition, radial profiles of flow quantities, like pressure and flow angle, are obtained to characterize the quality of flow field propagating from the turbine to the downstream exhaust diffuser, which has not been modeled here. Additional objectives are formulated based on the desirability of the turbine exit flow profile. The

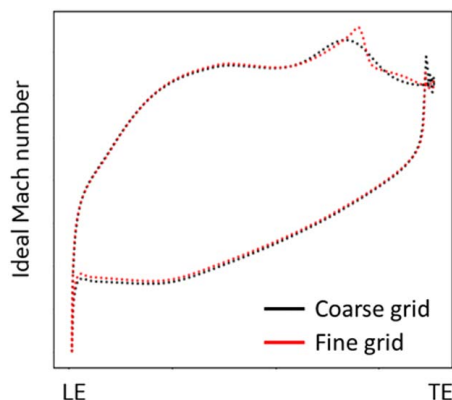


Fig. 8 Comparison between coarse and fine grid resolutions for the baseline case

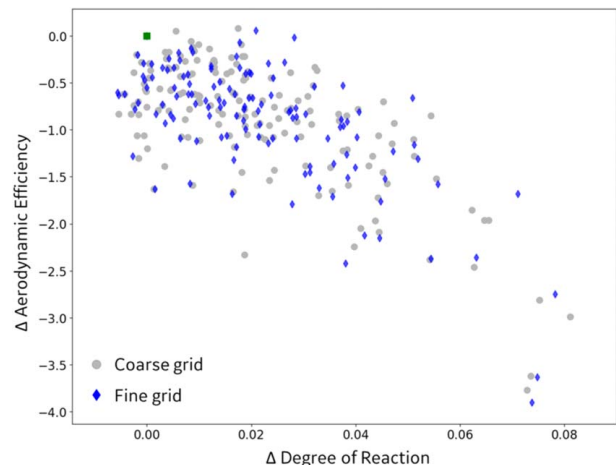


Fig. 9 DOE results for coarse and fine grid cases

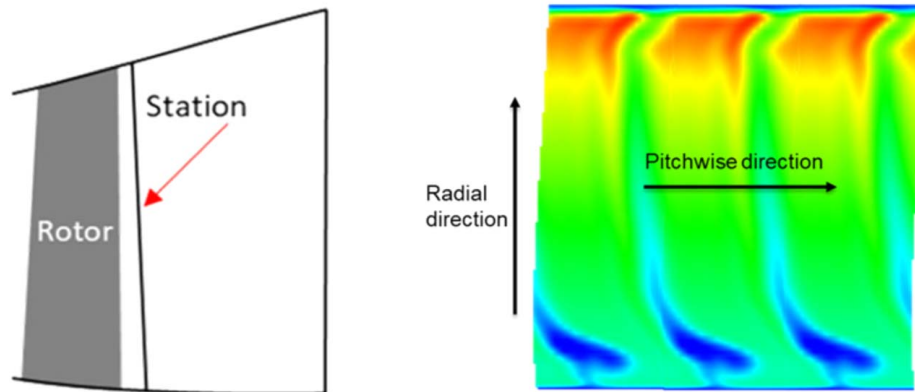


Fig. 10 Profile of absolute flow angle downstream of the rotor

focus is on the profiles of absolute total pressure and flow tangential angle, which are expected to have the most important effects on the diffuser performance. These flow quantities affect the momentum distribution going into the diffuser and the flow incidence to the downstream struts. Figure 10 illustrates the location where the profiles are extracted, downstream of the trailing edge. The right plot shows the rotor wake (repeated by applying periodicity) and shows the direction of the averaging. At each radial location, the quantities are averaged in the pitchwise direction over a single passage, so that the radial profiles include the effect of wakes and shock waves. The flow profiles are extracted downstream of the rotor trailing edge to assess the potential effect on the performance of the downstream exhaust diffuser.

A preliminary analysis of the CFD database showed that the rotor airfoil shape can impose consequential changes to the swirl profile, both in terms of mean value and variability, requiring characterization and objective control in the cINN.

To produce a turbine design which can couple with the diffuser with a satisfactory performance, we want to be able to control the behavior of pressure and swirl close to the end wall, in the boundary layer region, and at midspan. To understand what features drive diffuser performance, we leverage the diffuser CFD models and apply a subset of the DOE turbine exit flow profiles as diffuser inlet conditions. Representative merit criteria were extracted for the rotor profiles and correlated with the diffuser performance in terms of recovery factor. This approach is useful if we want to characterize the rotor exit profile in terms of scalar metrics. In addition, the framework has the capability to impose the profiles as vectors, point by point. This capability can be used if a precise inlet flow field is required at the inlet of the diffuser, and the description through scalar metrics is not sufficiently precise.

4.2 Forward Modeling. As discussed in the previous section, the design space of forward process is 85 dimensional. The output consists of both scalar and vector variables. The scalar variables consist of efficiency and pseudo-reaction. The vector outputs are exit flow profiles given by pressure ($\mathbb{R}^{100 \times 1}$) and swirl angle profiles ($\mathbb{R}^{100 \times 1}$), which are pressure and swirl angle measurements, respectively, at 100 discretized span-wise location. For the vector outputs, dimensionality reduction is carried out using Principal Component Analysis (PCA) to reduce the dimension of encoded profiles ($\mathbb{R}^{200 \times 1}$) to the lower dimension ($\mathbb{R}^{6 \times 1}$) represented by the PCA coefficients, capturing >90% energy of original variables. Next, all the scalar objectives and PCA coefficients of the vectors are used to build MFGP model using GEBHM.

In this work, we used multifidelity adaptive sampling [26] to pick designs and fidelity of CFD simulations to create the DOE for MFGP training. The new designs of the DOE are adaptively picked based on the cost ratio as well as the amount of uncertainty reduction associated with high- and low-fidelity simulations. With

the goal of achieving less than 10% of normalized root-mean-square error (nRMSE), a total of 249 low-fidelity and 231 high-fidelity CFD simulations were required. The parameters of MFGP are estimated using MCMC [24], which is a fully Bayesian approach to estimate the posterior distribution of the parameters based on the observed data and prior distribution. Therefore, it quantifies the uncertainty on the MFGP parameters associated with the sparse data and performs robustly in terms of predictive capability.

We estimated the accuracy with coefficient of determination (R^2) and nRMSE. nRMSE is root-mean-square error (RMSE) normalized by the difference between maximum and minimum values of output, i.e., $\text{nRMSE} = \text{RMSE} / (y_{\max} - y_{\min})$. To test the accuracy of the model, around 10% of high-fidelity data were hold-out to measure the R^2 and nRMSE. The overall accuracy in terms of R^2 and nRMSE of all the models of forward process is given in Table 2.

For the scalar objectives (efficiency and pseudo-reaction), nRMSE of less than 10% was achieved, which has been the requirement from the design standpoint. The model of the first four PCA coefficients achieved nRMSE of 10% or better. It must be noted that the first four PCA coefficients captures more than 80% variability of the vector profiles. The last two PCA coefficients (PCA-5 and PCA-6) are associated with high-frequency modes, which act very similar to noise. Although the PCA-5 and PCA-6 models are low in R^2 for validation data, the models are more than 80% accurate in terms of nRMSE. We also evaluated the accuracy of combination of all the PCA models to predict the vector profiles (absolute pressure and swirl angle). We estimate the error on vector profiles by calculating the maximum percentage deviation between predicted and true vector field for a given design. The histogram of the maximum percentage deviation for the validation data set is shown in Fig. 11. The models are within the maximum deviation of $\pm 2\%$ for the absolute pressure profiles and $\pm 8\%$ for the swirl angle profiles.

To estimate the cost saving in the forward modeling with MFGP and multifidelity adaptive sampling, we also trained SF GP models

Table 2 Accuracy metrics: normalized root-mean-squared error (nRMSE) and R^2 score for the aerodynamic efficiency, the pseudo-reaction, and the six PCA coefficients (for the pressure and the swirl angle profiles)

	R^2	nRMSE
Efficiency	0.792	0.094
Pseudo-reaction	0.909	0.083
PCA-1	0.965	0.063
PCA-2	0.920	0.069
PCA-3	0.850	0.103
PCA-4	0.781	0.109
PCA-5	0.357	0.199
PCA-6	0.389	0.169

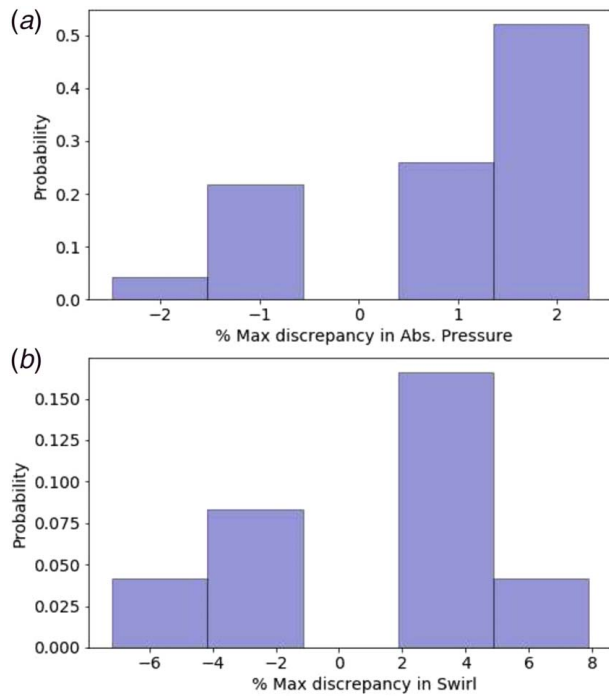


Fig. 11 Maximum percentage deviation of: (a) absolute pressure and (b) swirl angle profiles predicted by combination of PCA models of vector objective when compared to true values of validation data set of high-fidelity CFD simulations

with only high-fidelity data using random Latin hypercube sample, with sample size of 125, 175, and 200. Then, we evaluate the corresponding nRMSE with the same hold-out data, which were used for MFGP with multifidelity adaptive sampling. Based on the trend of nRMSE versus equivalent high-fidelity CFD analysis cost (No. of high-fidelity data + $(1/\text{cost ratio}) \times \text{No. of low-fidelity data}$), we estimate a cost saving of at least 35% in terms of computational cost associated with the forward modeling, as shown in Fig. 12.

4.3 Inverse Modeling. In this Study, there are 85 parameters considered as the input data for the cINN model. The output data consists of the aerodynamic efficiency, the pseudo-reaction, and the pressure and the swirl angle profiles with a total of 202 values for a given 3D blade. In this work, we pre-process the input data such that each parameter follows a standard normal distribution with mean 0 and variance 1. We also normalize the output data between zero and one. As mentioned earlier, we develop the inverse surrogate model that maps the noisy observations to the input data. The inverse mapping is confined to a broad prior distribution of the input data with which we train our cINN model.

Once the airfoil parameters for each of the samples S are predicted using Algorithm 2, we propagate these design samples through the forward surrogate model (GEBHM) to generate the outputs: the aerodynamic efficiency, the pseudo-reaction, the full pressure, and the swirl angle profiles, as shown in Fig. 13. Similar to the forward model, we consider R^2 and nRMSE to evaluate the developed inverse surrogate model on the test data $\{\mathbf{x}^i, \mathbf{y}^i\}_{i=1}^T$, where T is the total number of test data from the forward surrogate model that was unseen when training the cINN model. Note that once our inverse surrogate model is trained, we can solve many inverse problems with different observations. Here, we test our model with different observation data.

We train the inverse surrogate model with $N=60000$ training data ($\mathcal{D}$) that was generated from the GEBHM. We train the cINN model using the Adam optimizer [44] with the learning rate of $1e-5$, weight decay of $5e-7$, and mini-batch size of 16. The invertible network consists of $L=8$ invertible blocks, and in each

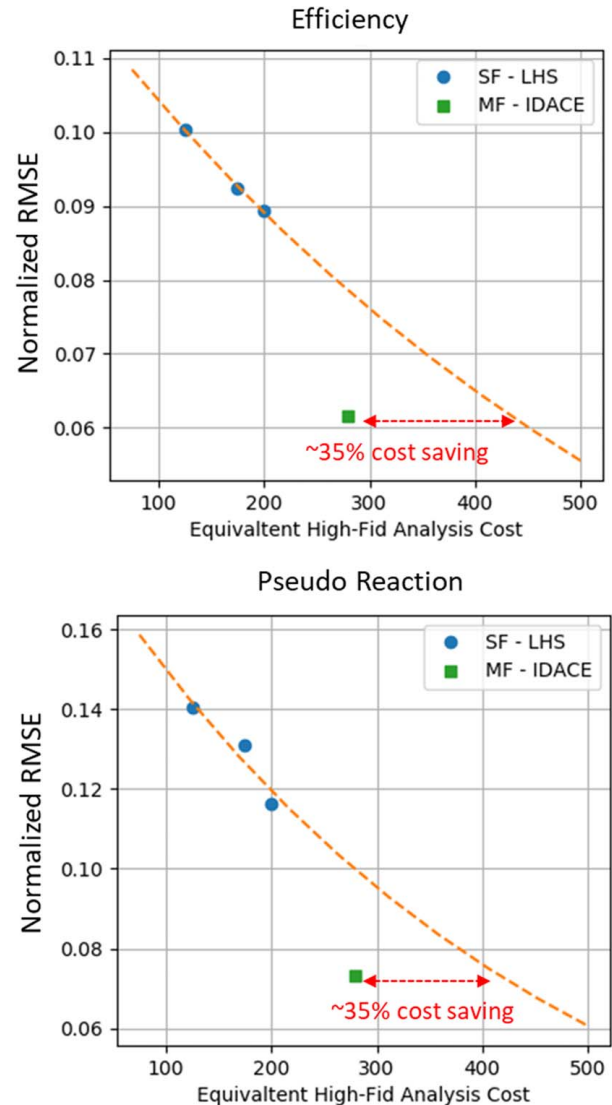


Fig. 12 Cost saving in the forward modeling using MFGP and multifidelity adaptive sampling

invertible block, the scale $s(\cdot)$ and the shift $t(\cdot)$ networks are fully connected layers that consist of three hidden layers with 256, 512, and 256 hidden units and employ dropout with a keep probability of 0.2. The conditioning network consists of fully connected network, which consists of four hidden layers with 400, 512, 640, and 896 hidden units, respectively. As mentioned earlier, both the conditioning network and the invertible network are trained in an end-to-end fashion. In this work, we train our model for 200 epochs and use a learning rate scheduler which drops ten times on the plateau of the mean negative log-likelihood. We show the mean negative log-likelihood (NLL) error during training for 200 epochs in Fig. 14. We observe that our inverse surrogate model converges at 120 epochs, and the learning rate scheduler drops the learning rate at epoch number 70.

In this work, we propagate the cINN samples $\{\hat{\mathbf{x}}^{(j)}\}_{j=1}^S$ through the forward surrogate model to generate the aerodynamic efficiency, the pseudo-reaction, and the pressure and the swirl angle profiles. Here, we consider 100 test data that were unseen during training the cINN model.

The ground truth versus the forward GEBHM surrogate solution for the samples generated using the cINN model is shown in Fig. 15(a) for the aerodynamic efficiency and Fig. 15(b) for pseudo-reaction. In each subplot, we show the mean μ of the predictions for the objectives mentioned earlier and the spread ($\mu \pm \sigma$). For

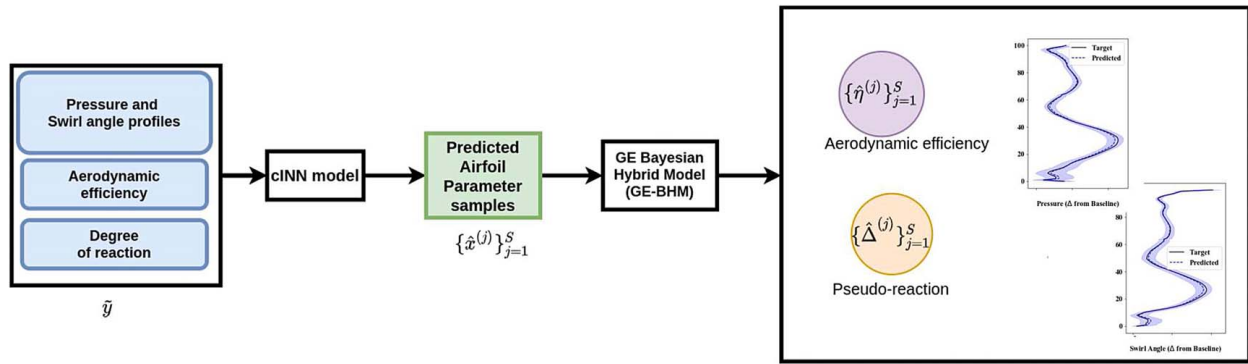


Fig. 13 Framework for postprocessing the samples generated from the cINN model using the GE Bayesian hybrid modeling. Once the airfoil parameters for each S samples are predicted, we propagate these design samples through the Bayesian hybrid modeling (BHM) model to generate the pressure and swirl angle profiles, aerodynamic efficiency, and pseudo-reaction.

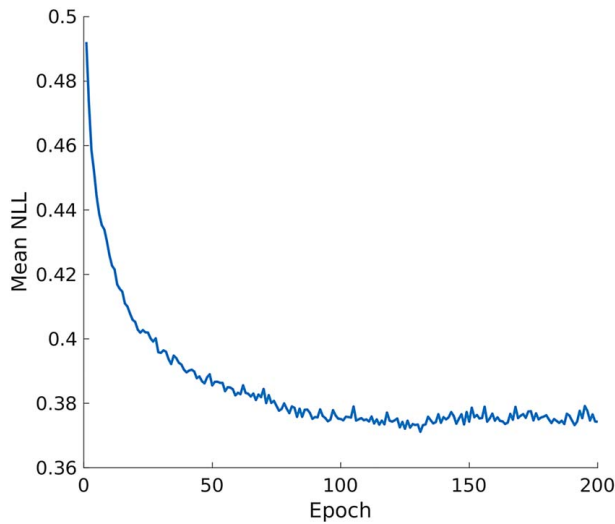


Fig. 14 Mean NLL loss during training cINN for 3D blade

both the objectives, we observe that in most of the test data, the mean μ of the predicted samples coincides with the ground truth values with a reasonable spread. The normalized root-mean-squared error and R^2 score for the aerodynamic efficiency, and the pseudo-reactions are tabulated in Table 3, and we see that the model can predict well for both the objectives with the accuracy above 95%.

In Fig. 16, we show the predicted pressure and swirl angle profile mean and the spread (95% confidence interval) for a test data y^* at the downstream of the rotor. We observe that the mean of the predicted pressure profile in Fig. 16(a), and the swirl angle profile shown in Fig. 16(b) almost coincides with the test data pressure (in Fig. 16(a)) and swirl angle profile (in Fig. 16(b)), respectively, with a reasonable spread shown in blue shaded area.

4.4 Overall Computational Fluid Dynamics Validation. To validate the 3D airfoil inverse modeling, a set of target points is provided as input to the PMI framework. The objectives are expressed as the target efficiency and the pseudo-reaction, together with rotor exit profiles as vectors, to retrieve a turbine design or a family of feasible designs. The selection of the objectives is intended to replicate a turbine design process, where a pseudo-reaction would be selected based on 1D design, and then, the turbine efficiency would be targeted or optimized. In addition, rotor exit flow characteristics are included as radial profiles of total pressure and swirl to account for the effect of the rotor design on the integrated system performance.

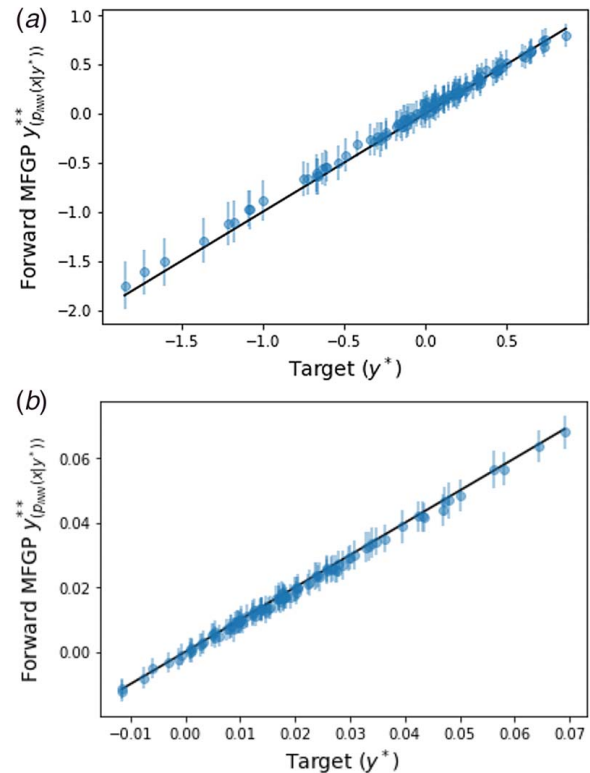


Fig. 15 The ground truth versus the forward BHM solution for the cINN model generated samples: (a) aerodynamic efficiency and (b) pseudo-reaction. The dot shows the mean of the predicted objectives, and the error bar shows the spread ($\mu \pm \sigma$) for each test data.

The process returns a distribution of design parameters, and five samples are selected for each target case to execute the CFD modeling and validate the objective values. In this process, the designs can be down selected based on additional constraints of geometrical properties. For this validation problem, the cross-sectional area, and

Table 3 Metrics for our proposed framework: normalized root-mean-squared error and R^2 score for the aerodynamic efficiency and the pseudo-reaction

	R^2	nRMSE
Efficiency	0.990	0.019
Pseudo-reaction	0.995	0.0124

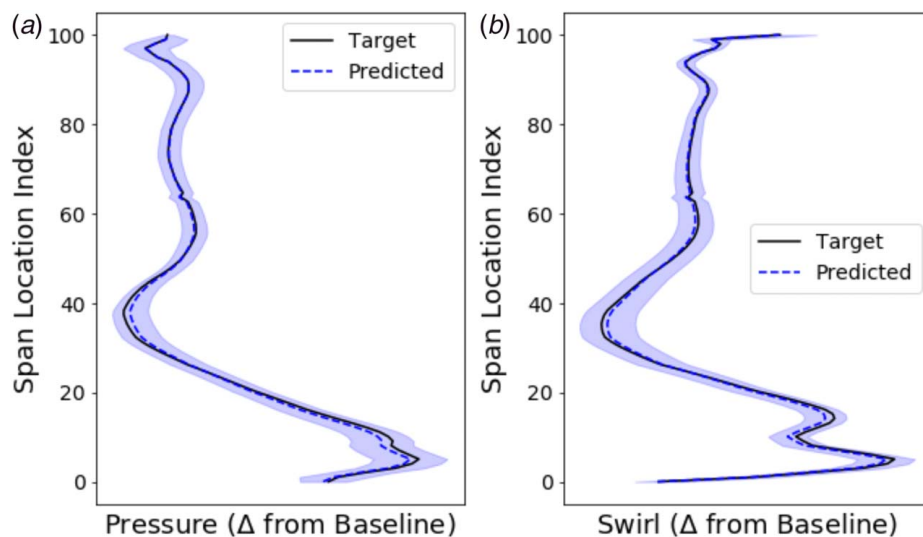


Fig. 16 The predicted pressure and swirl angle profile mean and the spread (95% confidence interval) along with the ground truth test data y . The solid curve is the target; the dashed curve is the mean of the predicted (a) pressure profile and (b) swirl angle profile, and the shaded area is 95% the confidence interval.

the maximum thickness at section 5 (close to midspan) are selected. The practical application of this feature is to retrieve a design able to ensure a certain performance, while also maintaining mechanical properties within a specified range. The designs, assessed through CFD, are in the range of desired efficiency with an uncertainty of approximately 0.2% points. The uncertainty decreases toward the region of high efficiency, which is a desirable feature for design applications where optimal designs are targeted. The uncertainty in the prediction of pseudo-reaction as delta from the baseline value is approximately 0.01 over the design space.

A detailed postprocessing is performed on a selection of these validation cases to analyze in detail rotor performance and the exit profiles. Figures 17 and 18 show the target profiles for two of the validation cases and the profiles retrieved from CFD for the cINN-based designs. Each figure shows the target profile as a solid black line; the target geometry also provides the efficiency

and reaction requested. The airfoils designed by the cINN are added to the charts in colored dashed lines. These airfoils all perform similarly to the baseline within uncertainty. All the profiles are shown as variations from a reference profile, calculated on the baseline design.

Two validation cases are shown in Figs. 17 and 18. These target profiles are selected to verify the cINN capabilities because the profiles are skewed toward the hub and the tip, respectively.

The two figures show radial profiles of absolute total pressure at the exit or the rotor at a station downstream of the trailing edge. The location where the profiles are extracted is sketched in Fig. 10. The quantities are averaged in the pitchwise direction, so the profiles represent variation of flow quantities along the rotor span. On the abscissa, the total pressure is reported as difference from a reference profile, as a percentage of the baseline profile itself. The ordinate shows the radial location, in percentage of the rotor span.

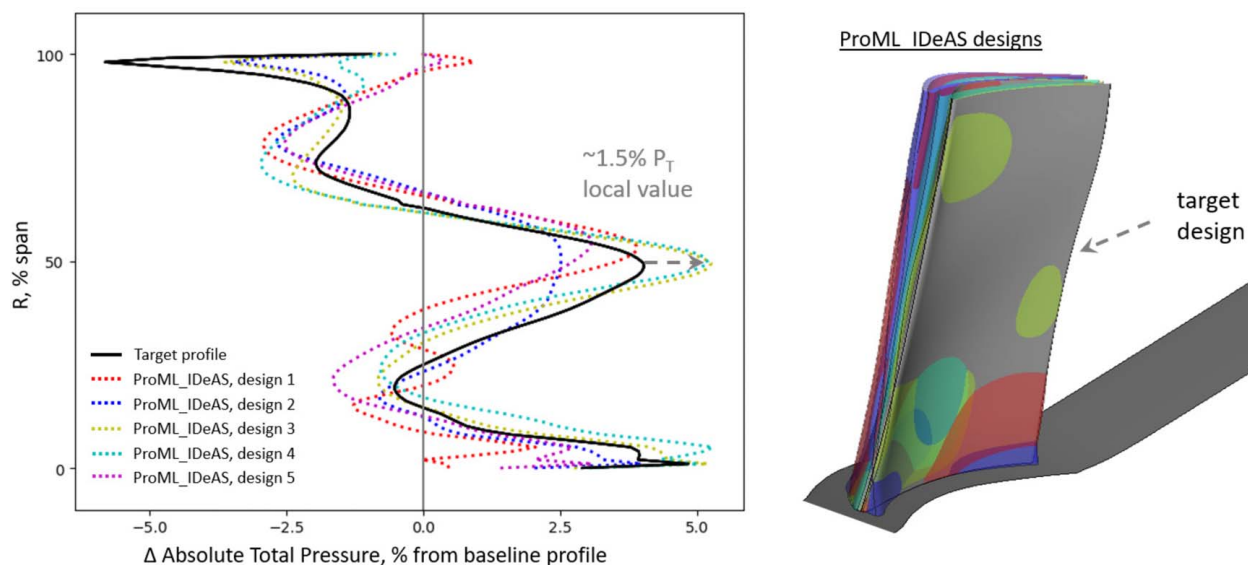


Fig. 17 Distribution of ideal Mach number along the airfoil for a given combination of target aerodynamic efficiency and reaction for a randomly selected test design

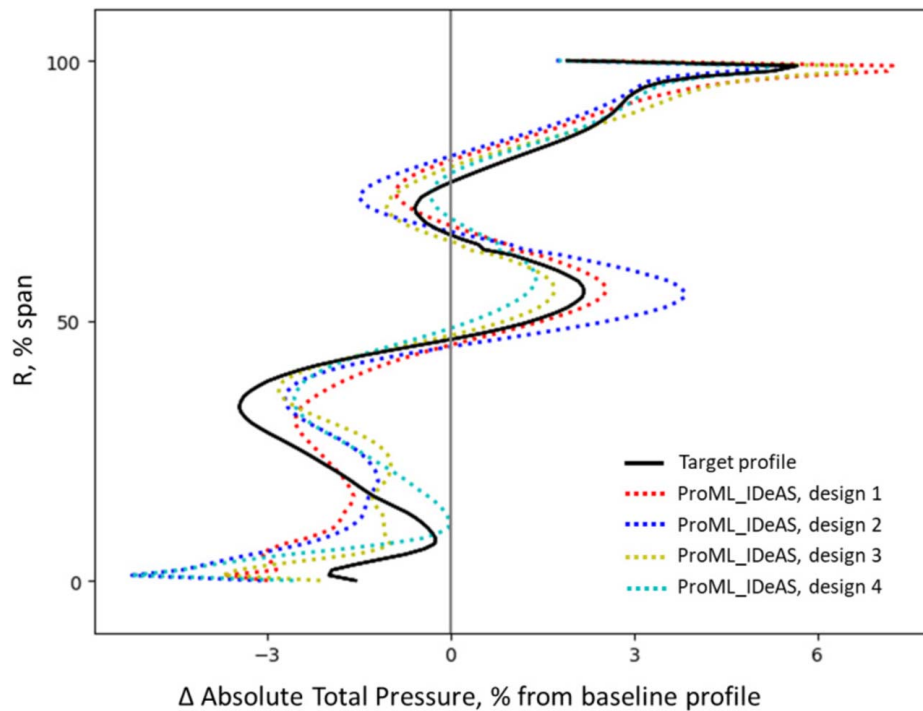


Fig. 18 Distribution of ideal Mach number along the airfoil for a given combination of target aerodynamic efficiency and reaction—case 25

The zero indicates the rotor hub, while the 100 is located at the turbine shroud.

The case shown in Fig. 17 has a profile below zero for most of the upper span, meaning that the target pressure is lower than the baseline pressure. At midspan and blade root, there is an increase of total pressure. The target profile is the solid black line. A set of designs, proposed by the framework, are analyzed with CFD and added to the chart. These results retrieved from CFD calculations on the cINN designs, marked with dashed lines, all show a hub pressure profile more energized than baseline and a higher pressure at midspan. Analogously, in Fig. 17, the target profile has higher pressure at the tip, and the framework is able to capture the trend and impose it to the retrieved geometry. In particular, the designs 2–4 are able to follow all the desired features, at hub, midspan, and shroud. The maximum deviation from target achieved by the framework is under 2% of the local pressure value. On the right side of Fig. 17, we show some of the blades retrieved by the PMI framework. Each blade is colored following the left plot, and the target blade is the gray surface. The blades are clearly different from the original blade, which is expected because the objective is set in terms of exit profile and not geometry. This method proposes a family of alternative designs to achieve analogous aerodynamic performance. Similarly, in Fig. 18, the second validation case is reported. The target profile in this case is pushing the total pressure at the shroud, while having a weaker hub. All the designs proposed by the framework, assessed with CFD, follow the desired trend as indicated by the colored dashed lines.

As demonstrated in the validation cases, the profiles are not restricted to an optimal solution. The cINN is trained using the designs generated by the forward model in the entire design space in which the forward model was trained. So, the cINN is capable of generating design solutions of the objectives that are possible within the design space (including the non-Pareto-optimal solutions). If a nonachievable target is provided to the cINN, the cINN tends to generate design solutions, which can perform close to the targets. To check how good the solutions of cINN are, designer can also run the forward model to evaluate the performance of the cINN solutions. The forward MFGP model

can provide a good estimate of the performance and quantify the uncertainty of cINN design solutions. Designer can use these information to pick designs for final evaluation using expensive CFD simulation.

It is also relevant to observe that the process generates a wide range of design options which ensure both performance and exit profiles. This feature will be key for a multidisciplinary design, when certain design options may not be feasible in other disciplines. Having multiple design options allows the designer to retain the ability to make decisions based on additional constraints that may not have been considered in the early stages of the design process.

5 Conclusion and Discussion

In this work, we demonstrate an “explicit” inverse aerodynamic design of industrial gas turbine blade airfoil using the Pro-ML IDEas (PMI) framework. In PMI, the explicit inverse design is modeled using conditional invertible neural network (cINN), which is a parameterized Bayesian bijective transform between design variables and engineering quantities of interest. To train the cINN model efficiently, data are cheaply generated using a probabilistic surrogate model of forward process. The surrogate of the forward process is built using a MFGP with adaptive sampling, which allows us to intelligently use limited data to build an accurate model.

For the inverse design modeling of 3D last stage blade, we used only 231 high-fidelity fine mesh and 249 low-fidelity coarse mesh CFD data to build an accurate forward MFGP model with accuracy of more than 90% for both the scalar and vector objectives. The training of final forward MFGP with 85 input dimension model takes around 2–3 h for each objective in a personal workstation without using parallel MCMC capability. In future, the trained MFGP model can be used for transfer learning for new creating new models for different turbine or operating conditions. The inverse model is trained using 60, 000 data points generated using the forward MFGP model. Based on our evaluation, the

cINN trained more than 98% accurate for the scalar objectives with respect to the forward MFGP model. The overall CFD validation results shows that the inverse design generated by INN are 0.2% points range of target efficiency and within an uncertainty of 0.01 for pseudo-reaction. In terms of vector objectives (pressure profiles), the design generated by the framework have a maximum deviation of 2% from target profiles.

Overall, the main computational cost associated with the PMI framework is linked with the cost and the amount of data generated to build an accurate forward generative model. Using a MFGP with adaptive sampling approach, resulted in savings of over 35%, in computational cost, over traditional approaches such as building GP models with one-shot design using Latin hypercube sampling. Training a cINN model for the blade aerodynamics problem using extensive and cheap data from the forward model takes an order of few hours of wall clock time. However, an inverse design query using the trained model to generate samples of design for a given target performance takes less than a second. In comparisons, other approaches such as Bayesian inversion approaches will take order of hours for each inversion query for the demonstrated problem. Therefore by using PMI framework, the savings in computational cost increase linearly with an increase in the number of inverse design queries.

As discussed in the previous section, the cINN can be queried for both Pareto-optimal and non-Pareto-optimal solutions. Although if target performance associate with Pareto-front is not known upfront, finding the Pareto-optimal design can be tricky and challenging. Alternatively, since, querying the cINN and forward models are very cheap ($O(\text{ms})$), designer can query multiple combination of extreme target performance (close to possible Pareto-front), find the design solution using cINN, and quantify the uncertainty associated with the design performance using the forward model.

The current framework mainly focuses on equality type objectives (and constraints). However, the inverse design with inequality can also be handled by the invertible network by querying the constraints values, which satisfies the constraint. For example, consider a design problem with target objective $y(\mathbf{x}) = y_{\text{target}}$ and inequality constraint $g(\mathbf{x}) \leq 0$. One can build an cINN with y and g as objectives. With the trained cINN, multiple queries to cINN can be made with target satisfying the constraints, such (y_{target}, g_1) , (y_{target}, g_2) , (y_{target}, g_3) , etc., where $g_i \leq 0$. These queries will generate multiple designs, which satisfies the design target as well as inequality constraint, from which the designer can pick solution based on their expertise and preference.

The main challenge of training cINN models are the number of data required for training such models. However, from the industrial standpoint, the data associated with design are generally sparse and expensive to generate. Our main contribution of this work came through overcoming this obstacle by developing the two-step PMI framework for training cINN for industrial challenging high-dimensional problems. In the first step of PMI framework, a fast and accurate forward model is developed using multifidelity data in an adaptive manner to decrease the data requirement. In the second step, the cINN is trained using cheap and large data generated using the fast forward model. In addition, we also extended the real NVP with a conditioning network in this work to handle challenges associated with deep generative models such as training stability and capturing sharp features in the underlying physical process.

In this work, we demonstrated the above framework on only the aerodynamic aspect of turbine blade design. Directions for future work include extending the framework to the multidisciplinary and aero-mechanics aspects of design, involving structural, modal, and flutter analysis. Moreover, the focus of demonstrating the capability of the proposed framework is on a challenging problem on IGT blades; however, the application of the framework is not limited to IGTs.

Acknowledgment

The information, data, or work presented herein was funded in part by the Advanced Research Projects Agency-Energy (ARPA-E), U.S. Department of Energy, under Award Number DE-AR0001204. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

Conflict of Interest

There are no conflicts of interest.

Data Availability Statement

The datasets generated and supporting the findings of this article are obtainable from the corresponding author upon reasonable request. Data provided by a third party listed in Acknowledgment.

References

- [1] Bui-Thanh, T., Damodaran, M., and Willcox, K., 2004, "Aerodynamic Data Reconstruction and Inverse Design Using Proper Orthogonal Decomposition," *AIAA J.*, **42**(8), pp. 1505–1516.
- [2] Dulikravich, G., and Baker, D., 1999, "Aerodynamic Shape Inverse Design Using a Fourier Series Method," 37th Aerospace Sciences Meeting and Exhibit, Reno, NV, Jan. 11–14, p. 185.
- [3] Obayashi, S., and Takanashi, S., 1996, "Genetic Optimization of Target Pressure Distributions for Inverse Design Methods," *AIAA J.*, **34**(5), pp. 881–886.
- [4] Dulikravich, G., 1995, "Shape Inverse Design and Optimization for Three-dimensional Aerodynamics," 33rd Aerospace Sciences Meeting and Exhibit, Reno, NV, Jan. 9–12, p. 695.
- [5] Bell, R., and Cedar, R., 1991, "An Inverse Method for the Aerodynamic Design of Three-Dimensional Aircraft Engine Nacelles," Third International Conference on Inverse Design Concepts and Optimization in Engineering Sciences, Washington, DC, Oct. 23–25.
- [6] Boselli, P., and Zangeneh, M., 2011, "An Inverse Design Based Methodology for Rapid 3D Multi-Objective/Multidisciplinary Optimization of Axial Turbines," Turbo Expo: Power for Land, Sea, and Air, Vol. 54679, Vancouver, British Columbia, Canada, June 6–10, pp. 1459–1468.
- [7] de Vito, L., Van den Braembussche, R. A., and Deconinck, H., 2003, "A Novel Two-Dimensional Viscous Inverse Design Method for Turbomachinery Blading," *ASME J. Turbomach.*, **125**(2), pp. 310–316.
- [8] Borges, J. E., 1990, "A Three-Dimensional Inverse Method for Turbomachinery: Part I Theory," *ASME J. Turbomach.*, **112**(3), pp. 346–354.
- [9] Nellippallil, A. B., Song, K. N., Goh, C. -H., Zagade, P., Gautham, B., Allen, J. K., and Mistree, F., 2017, "A Goal-Oriented, Sequential, Inverse Design Method for the Horizontal Integration of a Multistage Hot Rod Rolling System," *ASME J. Mech. Des.*, **139**(3), p. 031403.
- [10] Nickless, A., Rayner, P., Erni, B., and Scholes, R., 2018, "Comparison of the Genetic Algorithm and Incremental Optimisation Routines for a Bayesian Inverse Modelling Based Network Design," *Inverse Probl.*, **34**(5), p. 055006.
- [11] White, A., Mahadevan, S., Grey, Z., Schmucker, J., and Karl, A., 2021, "Efficient Calibration of a Turbine Disc Heat Transfer Model Under Uncertainty," *J. Thermophys. Heat. Transfer.*, **35**(2), pp. 234–244.
- [12] Hou, L., and Jiao, R. J., 2020, "Data-Informed Inverse Design by Product Usage Information: a Review, Framework and Outlook," *J. Intell. Manuf.*, **31**(3), pp. 529–552.
- [13] Bonaiuti, D., and Zangeneh, M., 2009, "On the Coupling of Inverse Design and Optimization Techniques for the Multiobjective, Multipoint Design of Turbomachinery Blades," *ASME J. Turbomach.*, **131**(2), p. 021014.
- [14] Roy, S., Das, R., and Saha, U. K., 2018, "An Inverse Method for Optimization of Geometric Parameters of a Savonius-Style Wind Turbine," *Energy. Convers. Manage.*, **155**, pp. 116–127.
- [15] Jin, K. H., McCann, M. T., Froustey, E., and Unser, M., 2017, "Deep Convolutional Neural Network for Inverse Problems in Imaging," *IEEE Trans. Image Process.*, **26**(9), pp. 4509–4522.
- [16] Chen, H., Zhang, Y., Kalra, M. K., Lin, F., Chen, Y., Liao, P., Zhou, J., and Wang, G., 2017, "Low-Dose CT With a Residual Encoder-Decoder Convolutional Neural Network," *IEEE Trans. Medical Imaging*, **36**(12), pp. 2524–2535.
- [17] Adler, J., and Öktem, O., 2017, "Solving Ill-Posed Inverse Problems Using Iterative Deep Neural Networks," *Inverse Probl.*, **33**(12), p. 124007.
- [18] Adler, J., and Öktem, O., 2018, "Learned Primal-Dual Reconstruction," *IEEE Trans. Medical Imaging*, **37**(6), pp. 1322–1332.
- [19] Hauptmann, A., Lucka, F., Betcke, M., Huynh, N., Adler, J., Cox, B., Beard, P., Ourselin, S., and Arridge, S., 2018, "Model-Based Learning for Accelerated, Limited-View 3-d Photoacoustic Tomography," *IEEE Trans. Med. Imaging*, **37**(6), pp. 1382–1393.

- [20] Mosser, L., Dubrule, O., and Blunt, M. J., 2020, "Stochastic Seismic Waveform Inversion Using Generative Adversarial Networks as a Geological Prior," *Math. Geosci.*, **52**(1), pp. 53–79.
- [21] Lunz, S., Öktem, O., and Schönlieb, C.-B., 2018, "Adversarial Regularizers in Inverse Problems," *Advances in Neural Information Processing Systems*, Montréal, Canada, Dec. 3–8, pp. 8507–8516.
- [22] Laloy, E., Héroult, R., Jacques, D., and Linde, N., 2018, "Training-Image Based Geostatistical Inversion Using a Spatial Generative Adversarial Neural Network," *Water. Resour. Res.*, **54**(1), pp. 381–406.
- [23] Ghosh, S., Padmanabha, G. A., Peng, C., Atkinson, S., Andreoli, V., Pandita, P., Vandeputte, T., Zabarar, N., Wang, L., and Khan, G., 2021, "Pro-ml Ideas: A Probabilistic Framework for Explicit Inverse Design Using Invertible Neural Network," *AIAA Scitech 2021 Forum*, Virtual Online, Jan. 11–15, p. 0465.
- [24] Ghosh, S., Pandita, P., Atkinson, S., Subber, W., Zhang, Y., Kumar, N. C., Chakrabarti, S., and Wang, L., 2020, "Advances in Bayesian Probabilistic Modeling for Industrial Applications," *ASCE-ASME J. Risk and Uncert in Eng. Syst. Part B Mech. Eng.*, **6**(3), p. 030904.
- [25] Pandita, P., Tsilifis, P., Ghosh, S., and Wang, L., 2021, "Scalable Fully Bayesian Gaussian Process Modeling and Calibration With Adaptive Sequential Monte Carlo for Industrial Applications," *ASME J. Mech. Des.*, **143**(7), p. 074502.
- [26] Ghosh, S., Kristensen, J., Zhang, Y., Subber, W., and Wang, L., 2019, "A Strategy for Adaptive Sampling of Multi-Fidelity Gaussian Processes to Reduce Predictive Uncertainty," *ASME 2019 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Anaheim, CA, Aug. 18–21, American Society of Mechanical Engineers Digital Collection.
- [27] Pandita, P., Bilonis, I., and Panchal, J., 2016, "Extending Expected Improvement for High-Dimensional Stochastic Optimization of Expensive Black-Box Functions," *ASME J. Mech. Des.*, **138**(11), p. 111412.
- [28] Pandita, P., Bilonis, I., Panchal, J., Gautham, B., Joshi, A., and Zagade, P., 2018, "Stochastic Multiobjective Optimization on a Budget: Application to Multipass Wire Drawing With Quantified Uncertainties," *Int. J. Uncertainty Quantif.*, **8**(3), p. 233.
- [29] Pandita, P., Tsilifis, P., Awalgankar, N. M., Bilonis, I., and Panchal, J., 2021, "Surrogate-Based Sequential Bayesian Experimental Design Using Non-Stationary Gaussian Processes," *Comput. Methods. Appl. Mech. Eng.*, **385**, p. 114007.
- [30] Hennig, P., and Schuler, C. J., 2012, "Entropy Search for Information-Efficient Global Optimization," *J. Machine Learning Res.*, **13**(6), pp. 1809–1837.
- [31] Rasmussen, C. E., and Williams, C. K. I., 2006, *Gaussian Processes for Machine Learning*, MIT Press, Cambridge, MA.
- [32] Kennedy, M., and O'Hagan, A., 2001, "Bayesian Calibration of Computer Models (With Discussion)," *J. R. Statist. Soc. (Ser. B)*, **68**(3), pp. 425–464.
- [33] Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y., 2020, "Generative Adversarial Nets," *Commun. ACM*, **63**(11), pp. 139–144.
- [34] Kingma, D. P., and Welling, M., 2014, "Auto-Encoding Variational Bayes," 2nd International Conference on Learning Representations. ICLR Conference Track Proceedings, Banff, AB, Canada, Apr. 14–16.
- [35] Isola, P., Zhu, J.-Y., Zhou, T., and Efros, A. A., 2017, "Image-to-Image Translation With Conditional Adversarial Networks," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, Honolulu, HI, July 21–26, pp. 1125–1134.
- [36] Mirza, M., and Osindero, S., 2014, "Conditional Generative Adversarial Nets," Preprint arXiv:1411.1784.
- [37] Sohn, K., Lee, H., and Yan, X., 2015, "Learning Structured Output Representation Using Deep Conditional Generative Models," *Advances in Neural Information Processing Systems*, Montreal, Canada, Dec. 7–12, pp. 3483–3491.
- [38] Ardizzone, L., Lüth, C., Kruse, J., Rother, C., and Köthe, U., 2019, "Guided Image Generation With Conditional Invertible Neural Networks," Preprint arXiv:1907.02392.
- [39] Dinh, L., Sohl-Dickstein, J., and Bengio, S., 2017, "Density Estimation Using Real NVP," *ICLR Conference*, Toulon, France, Apr. 24–26.
- [40] Padmanabha, G. A., and Zabarar, N., 2021, "Solving Inverse Problems Using Conditional Invertible Neural Networks," *J. Comp. Phys.*, **433**, p. 110194.
- [41] Geneva, N., and Zabarar, N., 2020, "Multi-Fidelity Generative Deep Learning Turbulent Flows," *Found. Data Sci.*, **2**(4), pp. 391–428.
- [42] Zhu, Y., Zabarar, N., Koutsourelakis, P.-S., and Perdikaris, P., 2019, "Physics-Constrained Deep Learning for High-Dimensional Surrogate Modeling and Uncertainty Quantification Without Labeled Data," *J. Comput. Phys.*, **394**, pp. 56–81.
- [43] Maas, A. L., Hannun, A. Y., and Ng, A. Y., 2013, "Rectifier Nonlinearities Improve Neural Network Acoustic Models," *Proc. ICML*, Vol. 30, Atlanta, GA, June 16–21, Citeseer, p. 3.
- [44] Kingma, D. P., and Ba, J., 2015, "Adam: A Method for Stochastic Optimization," *ICLR Conference*, San Diego, CA, May 7–9.